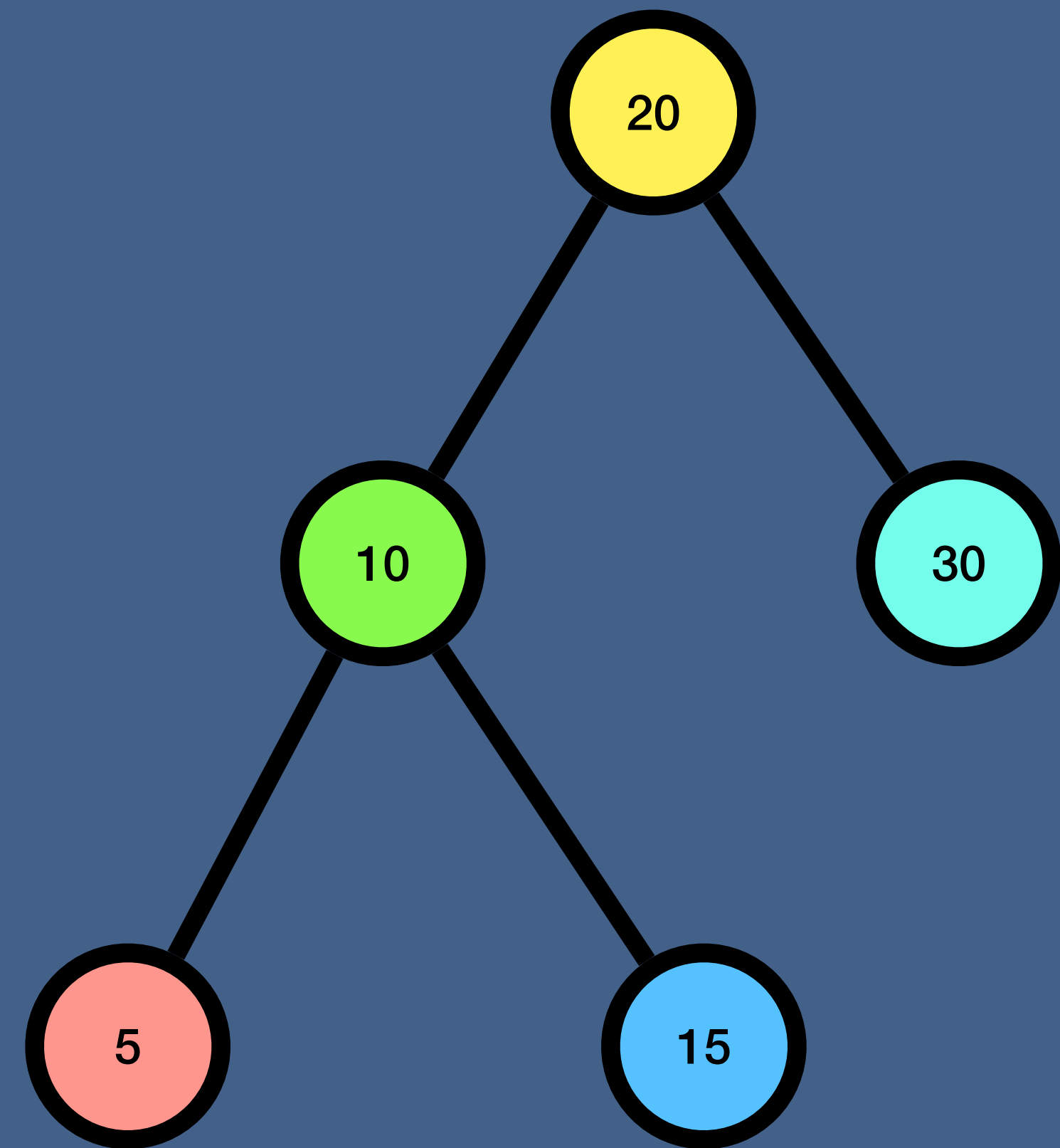
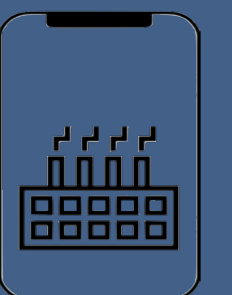
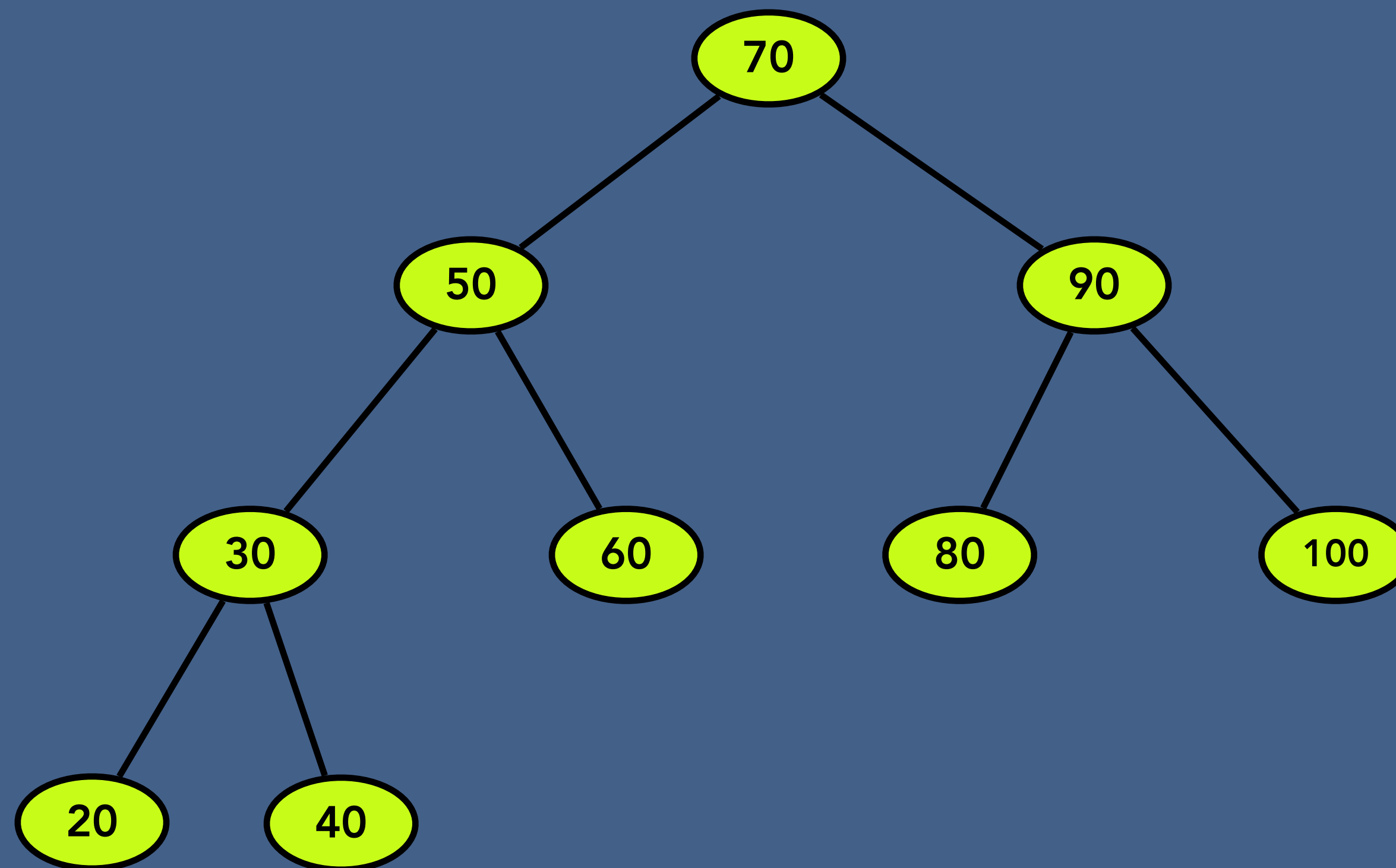


Binary Search Tree



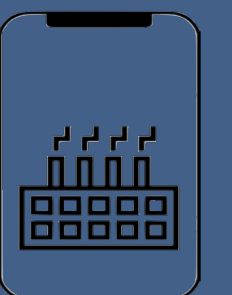
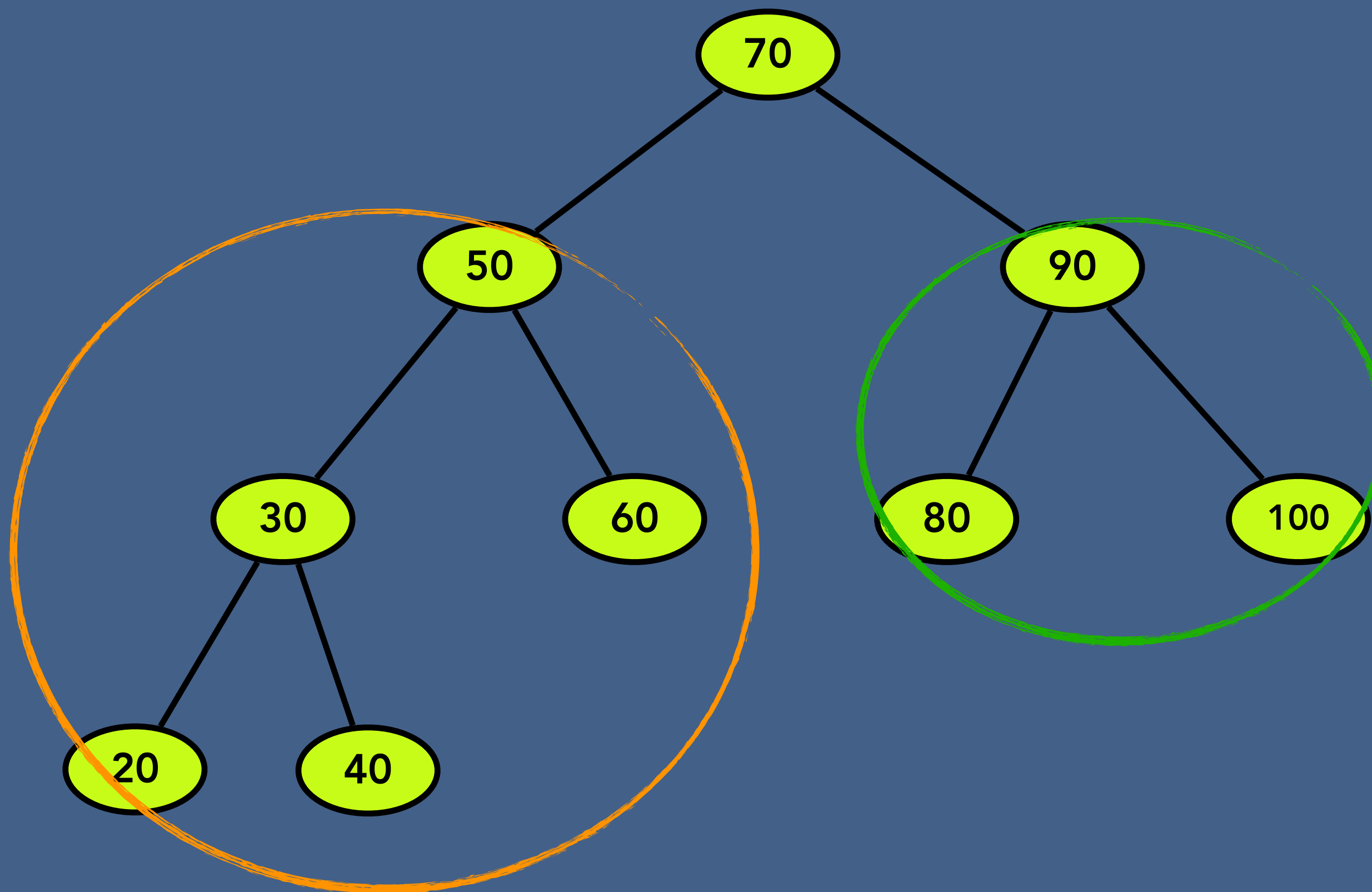
What is a Binary Search Tree?

- In the left subtree the value of a node is less than or equal to its parent node's value.
- In the right subtree the value of a node is greater than its parent node's value



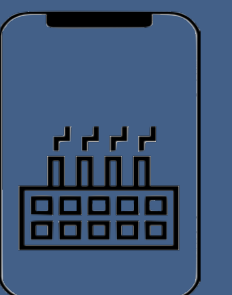
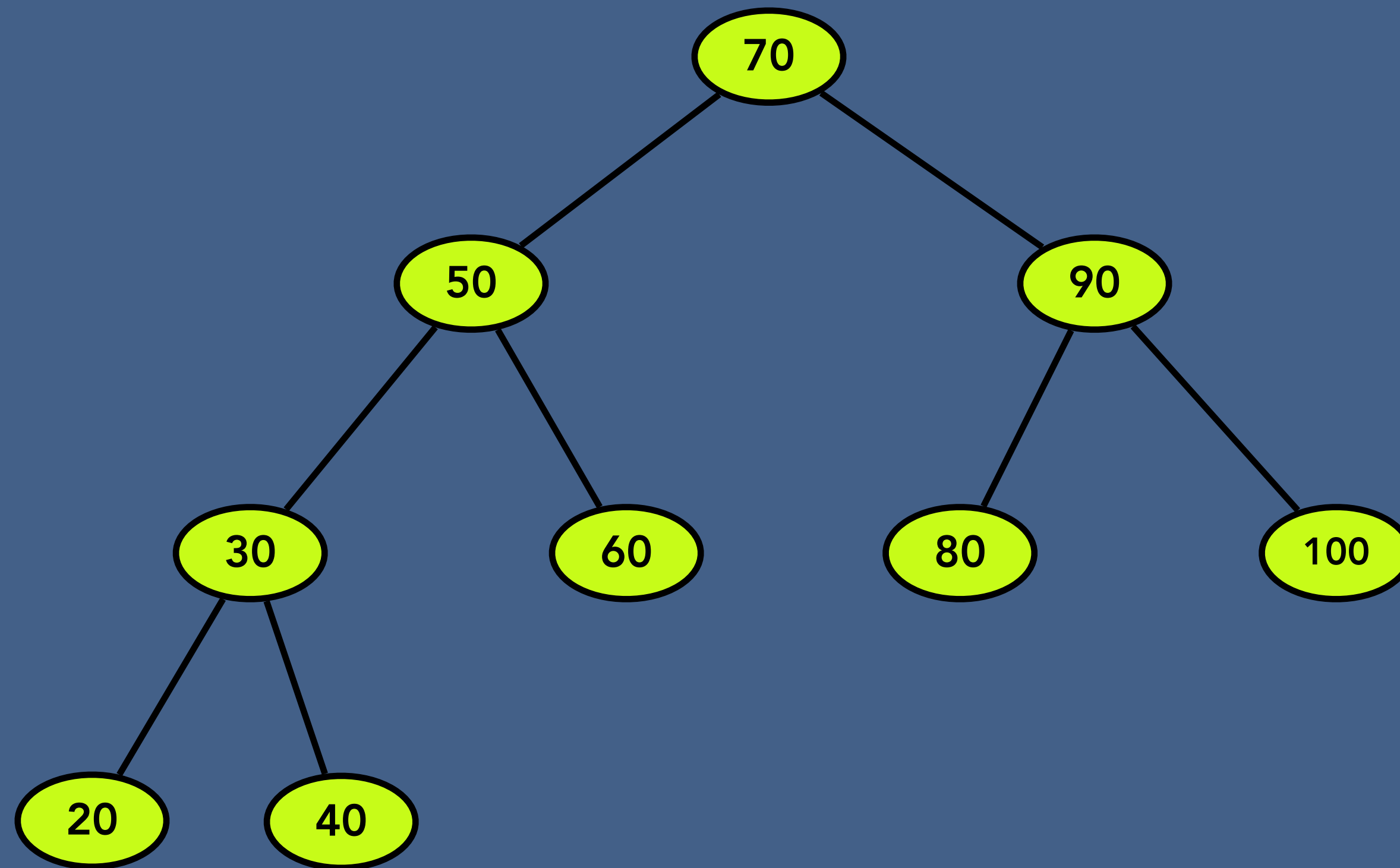
What is a Binary Search Tree?

- In the left subtree the value of a node is less than or equal to its parent node's value.
- In the right subtree the value of a node is greater than its parent node's value



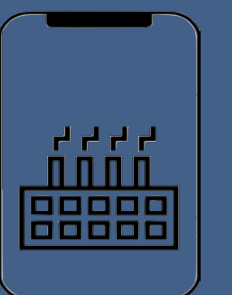
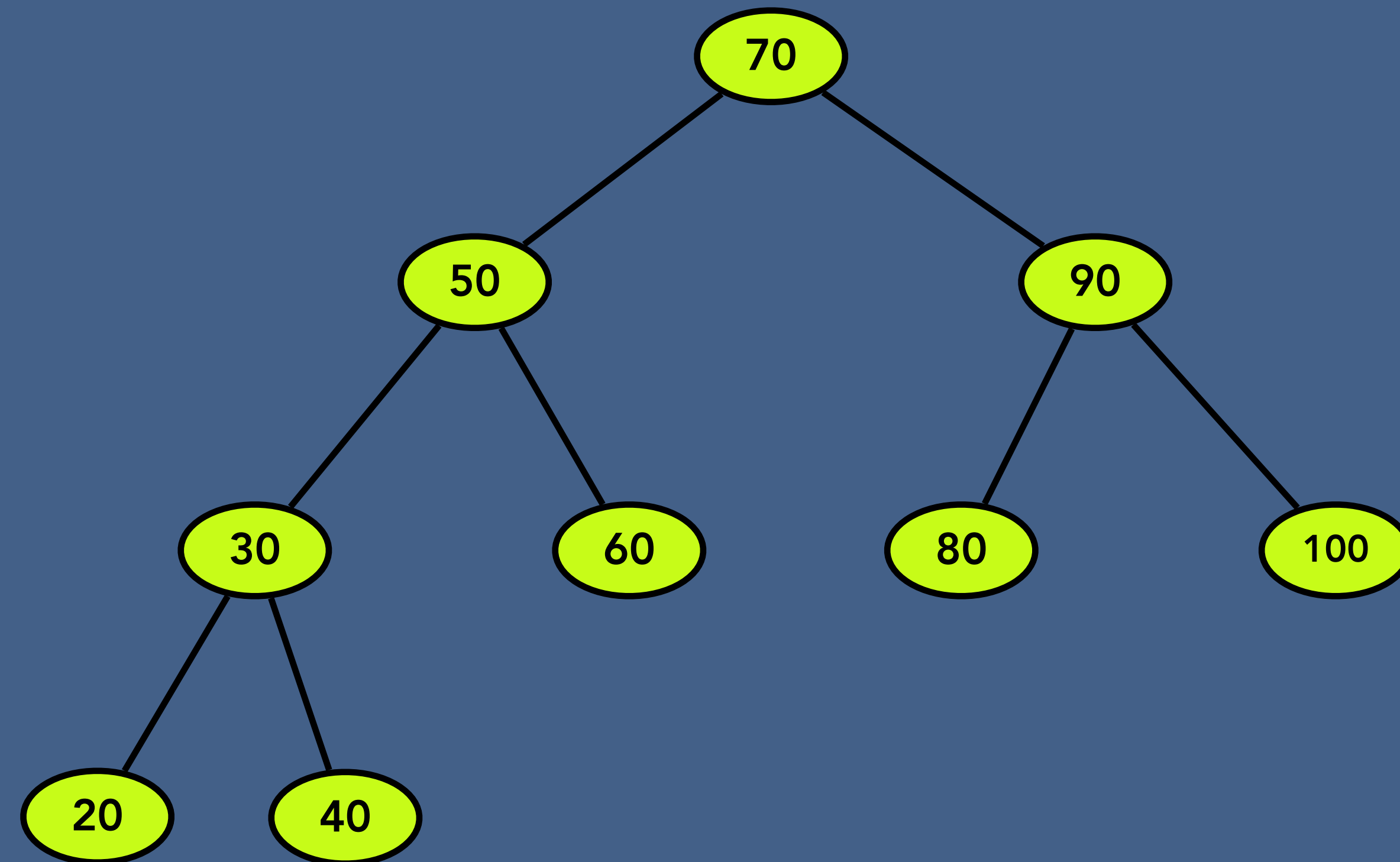
Why Binary Search Tree?

- It performs faster than Binary Tree when inserting and deleting nodes



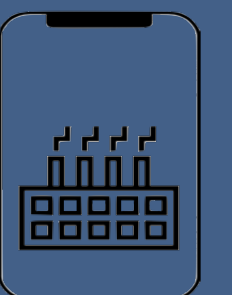
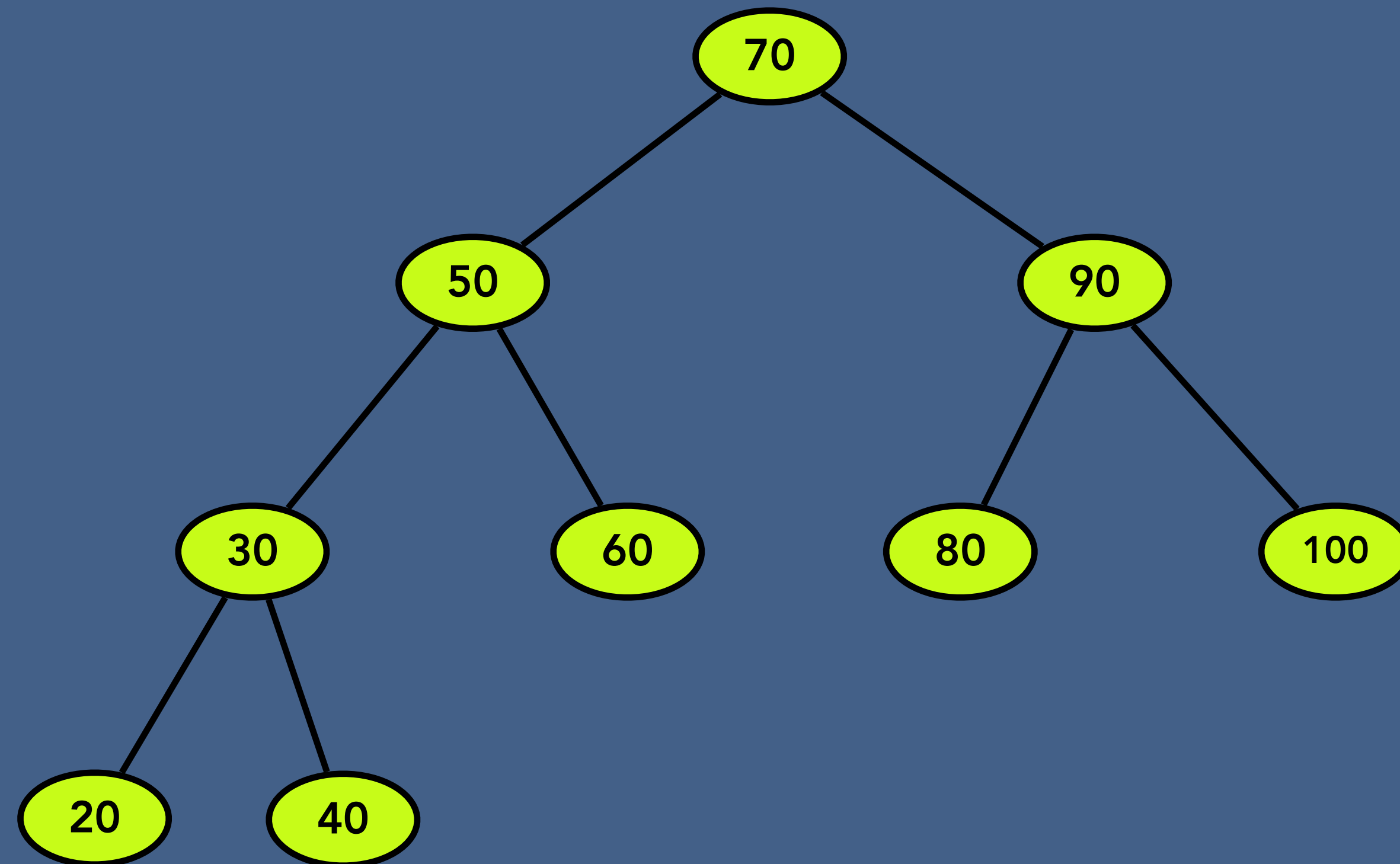
Common Operations on Binary Search Tree

- Creation of BST
- Insertion of a node
- Deletion of a node
- Search for a value
- Traverse all nodes
- Deletion of BST

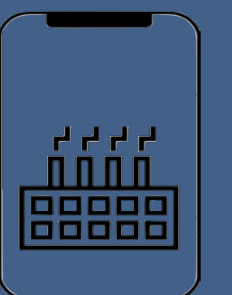
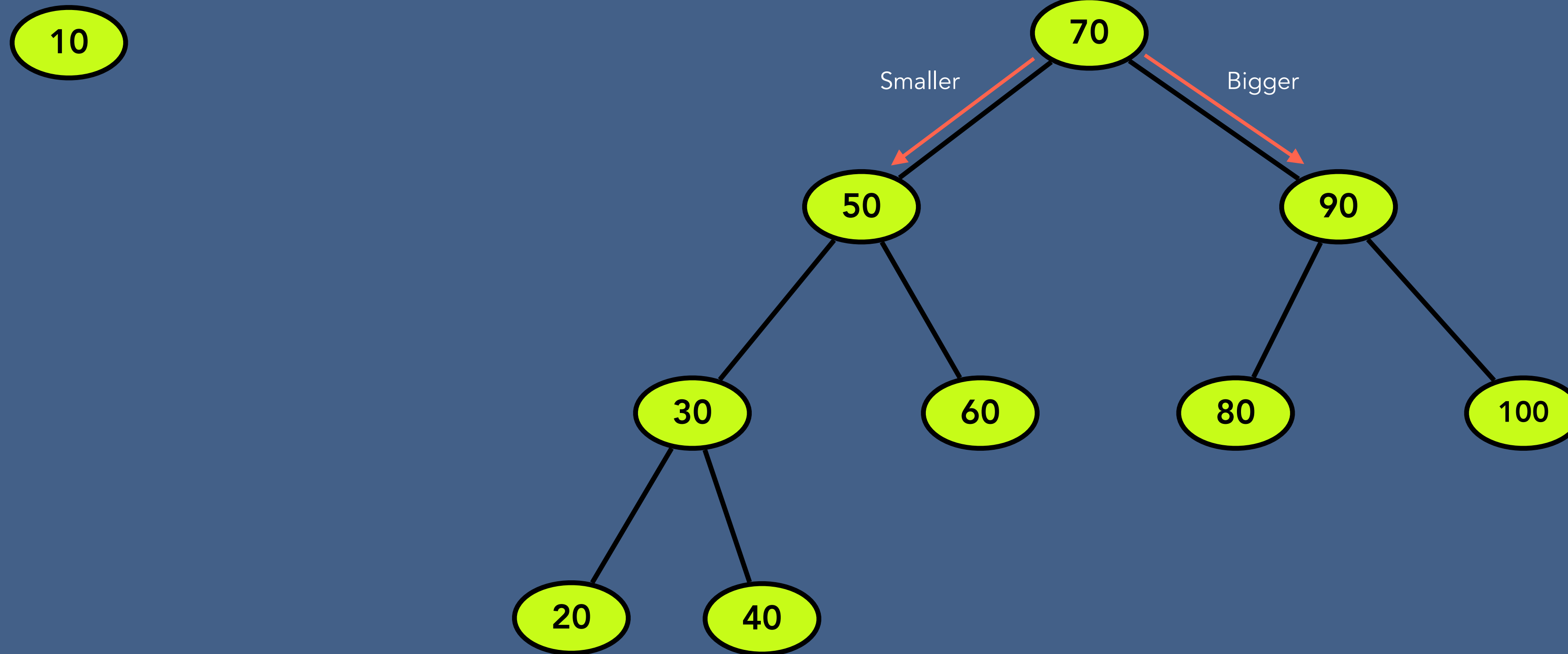


Create Binary Search Tree

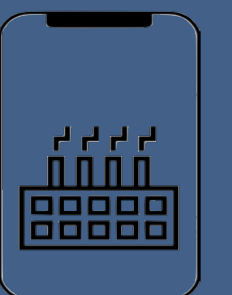
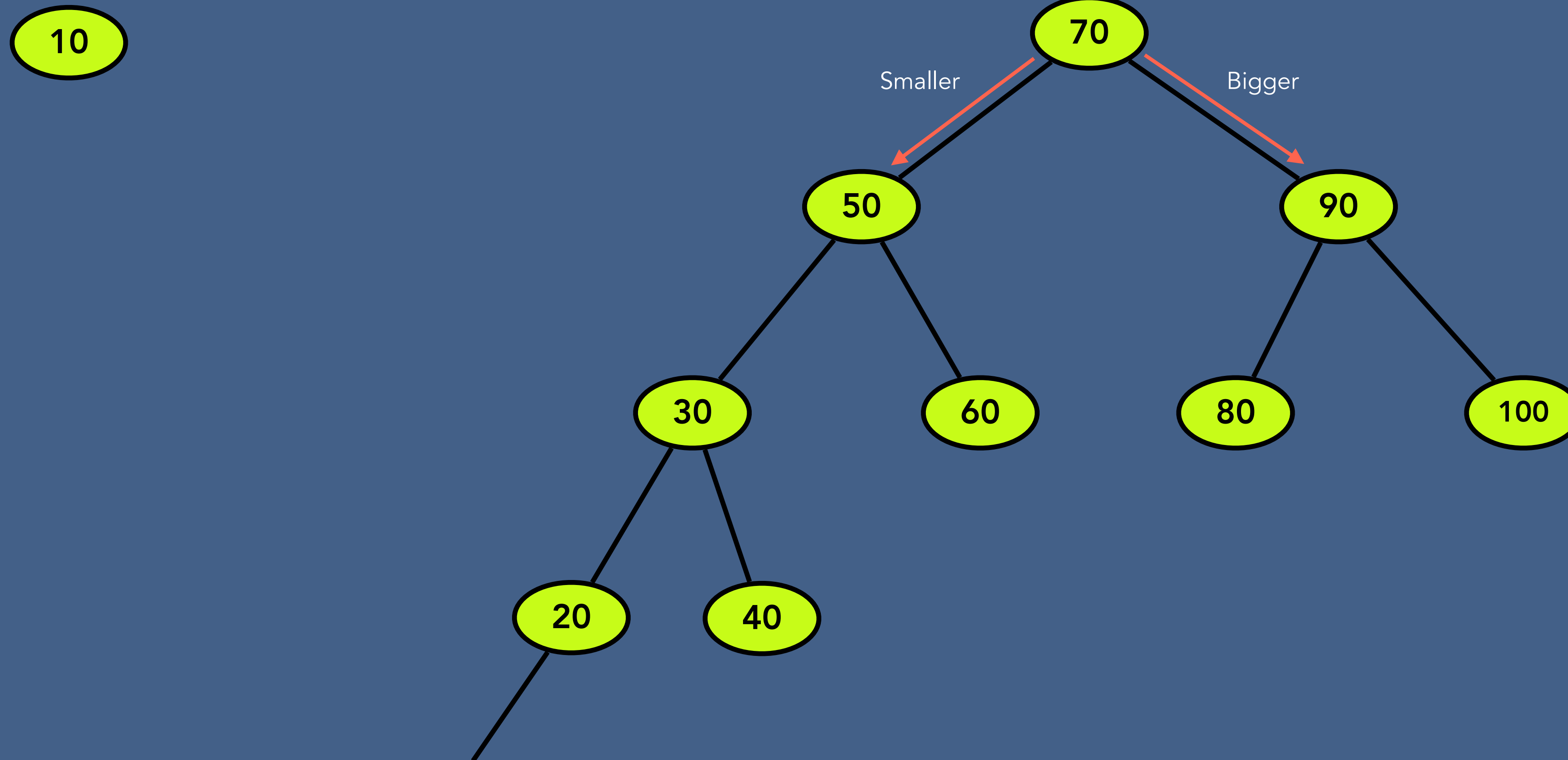
```
newBST = BST()
```



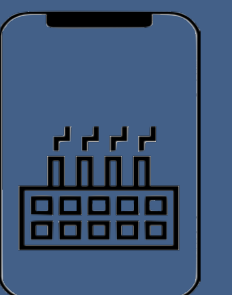
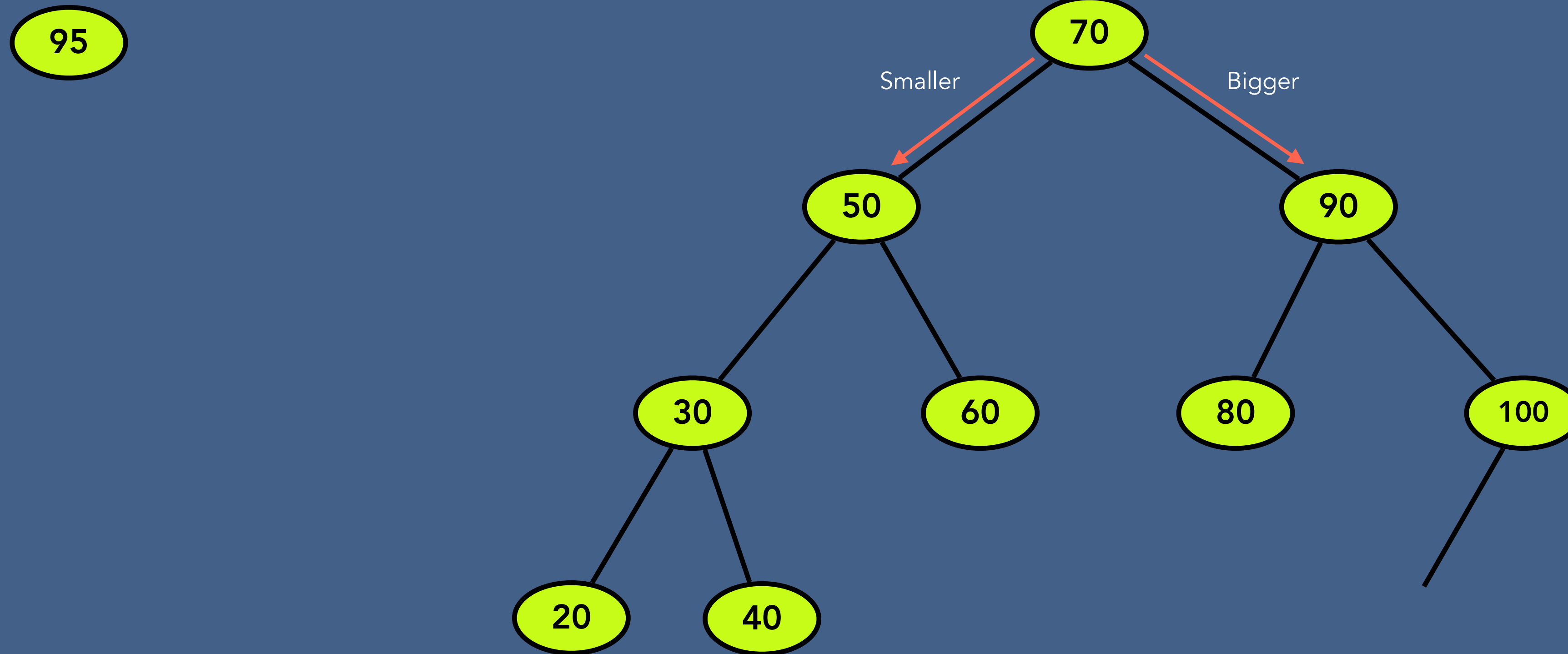
Binary Search Tree - Insert a Node



Binary Search Tree - Insert a Node



Binary Search Tree - Insert a Node



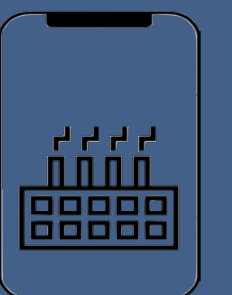
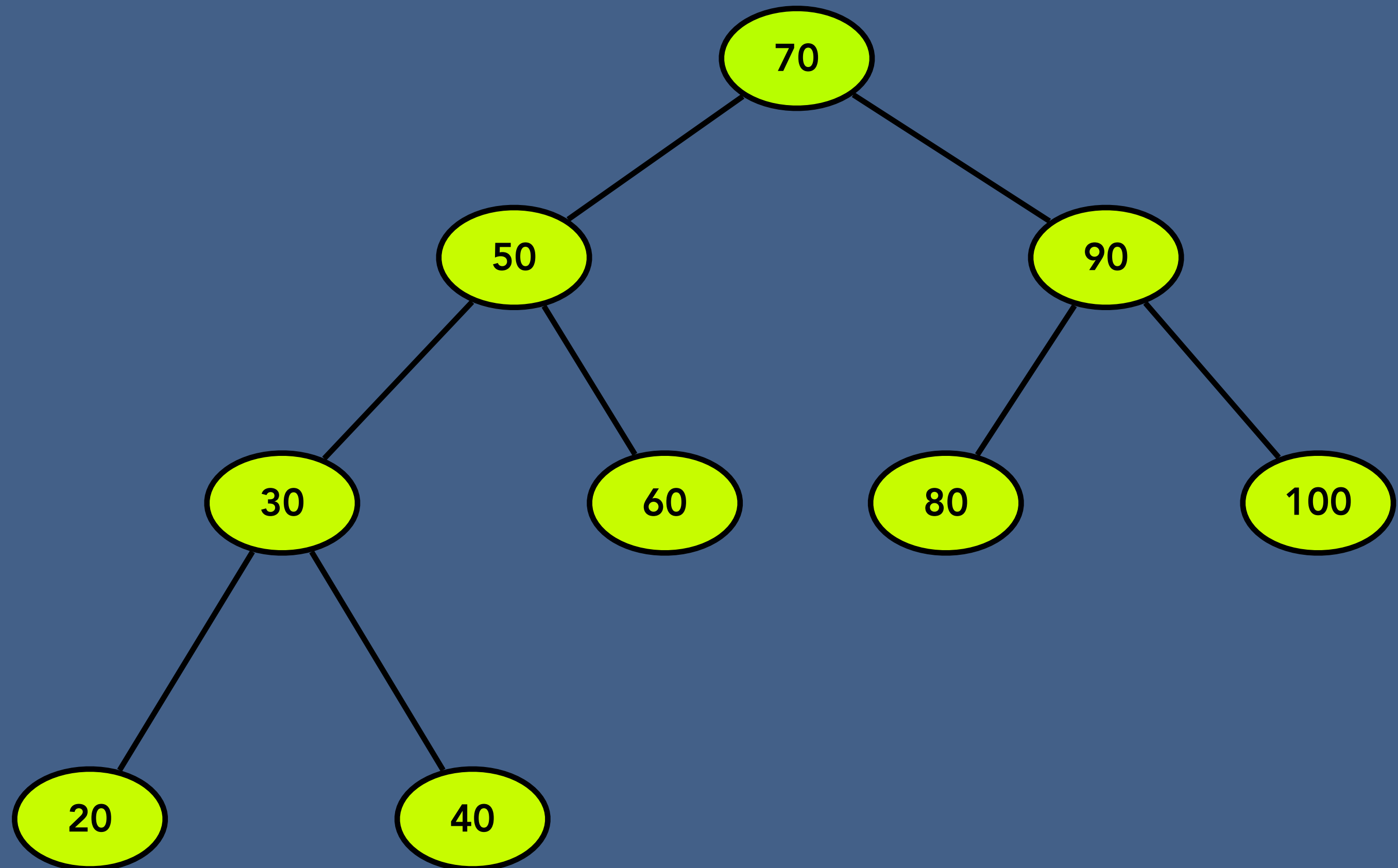
Binary Search Tree - Traversal

Depth first search

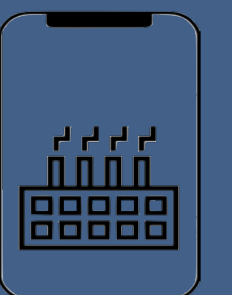
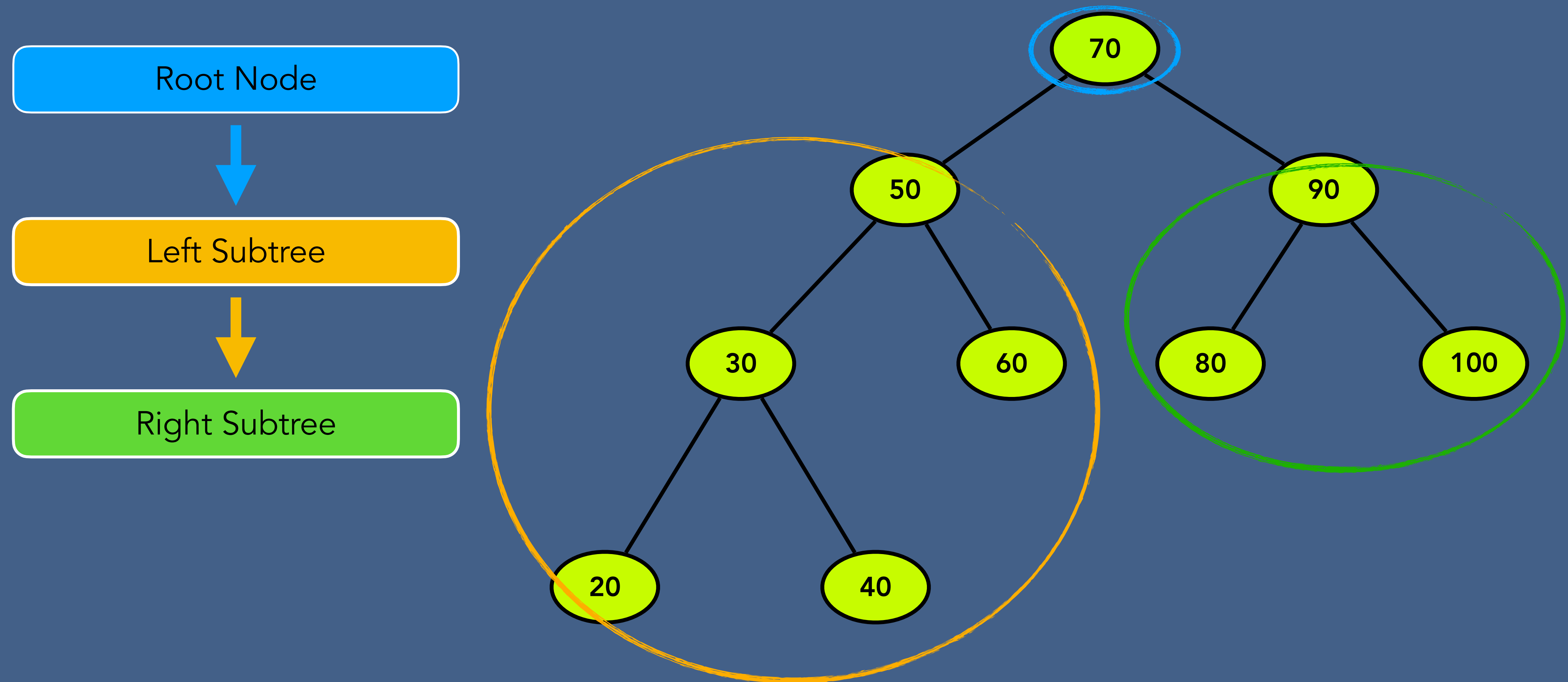
- Preorder traversal
- Inorder traversal
- Post order traversal

Breadth first search

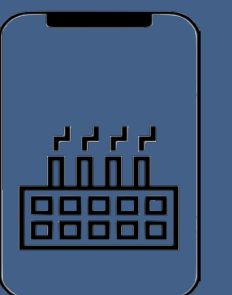
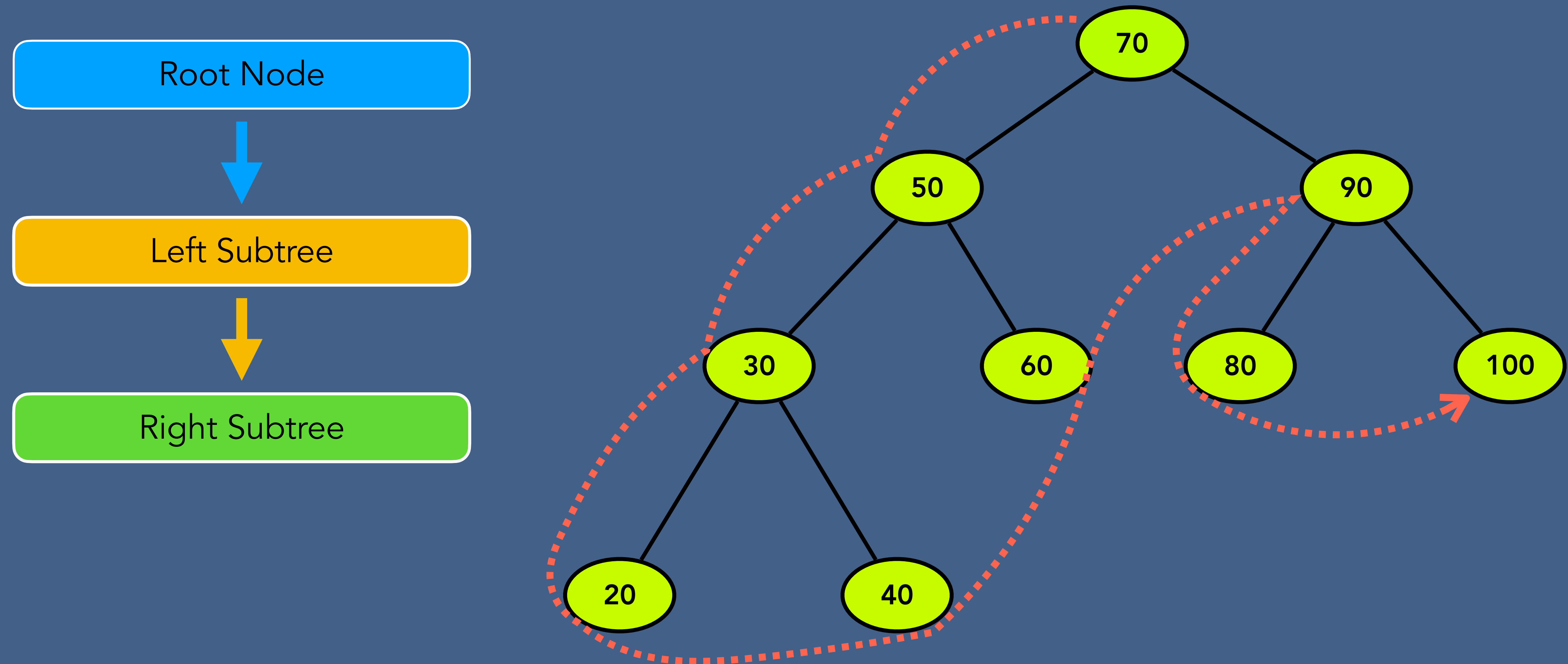
- Level order traversal



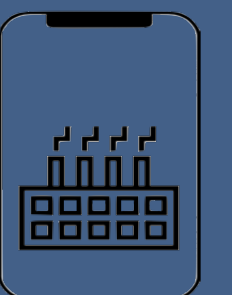
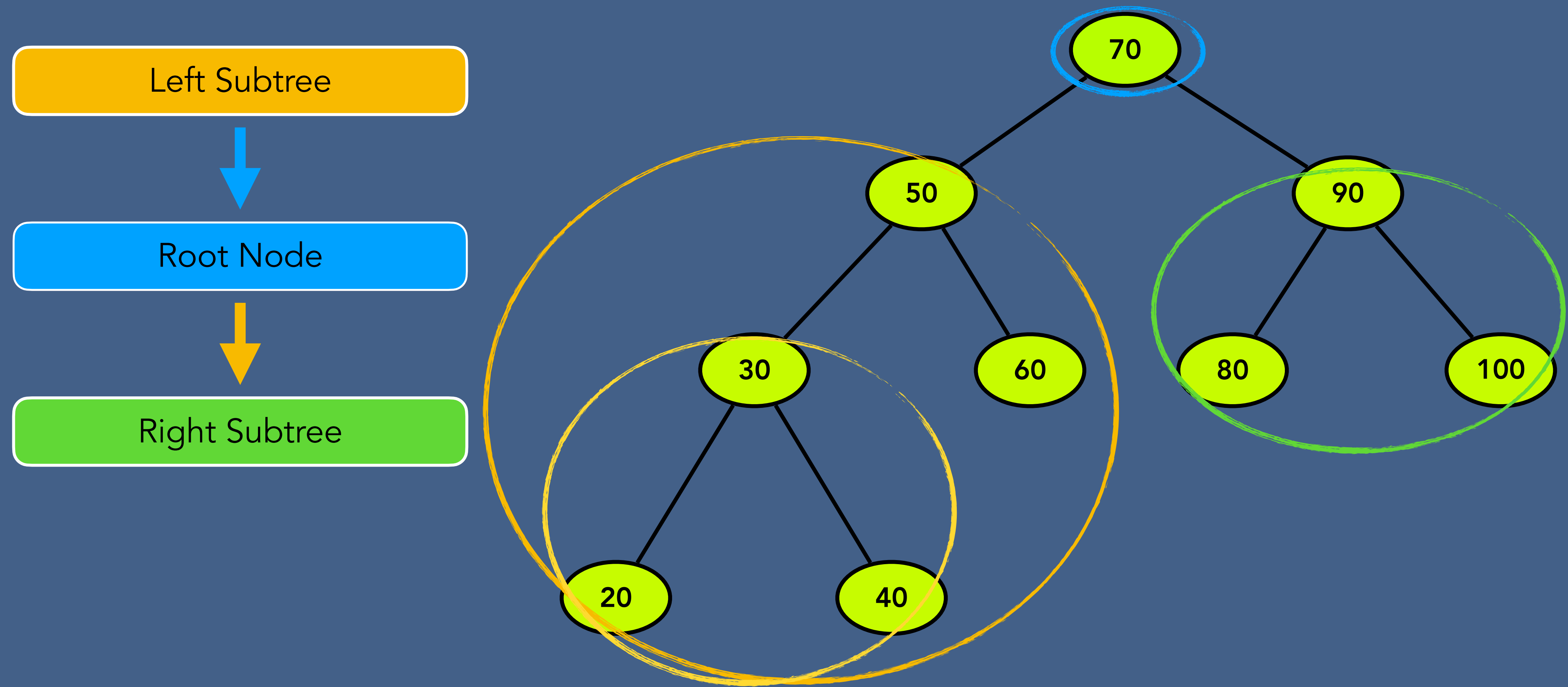
Binary Search Tree - PreOrder Traversal



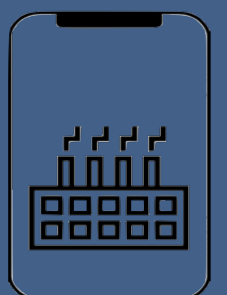
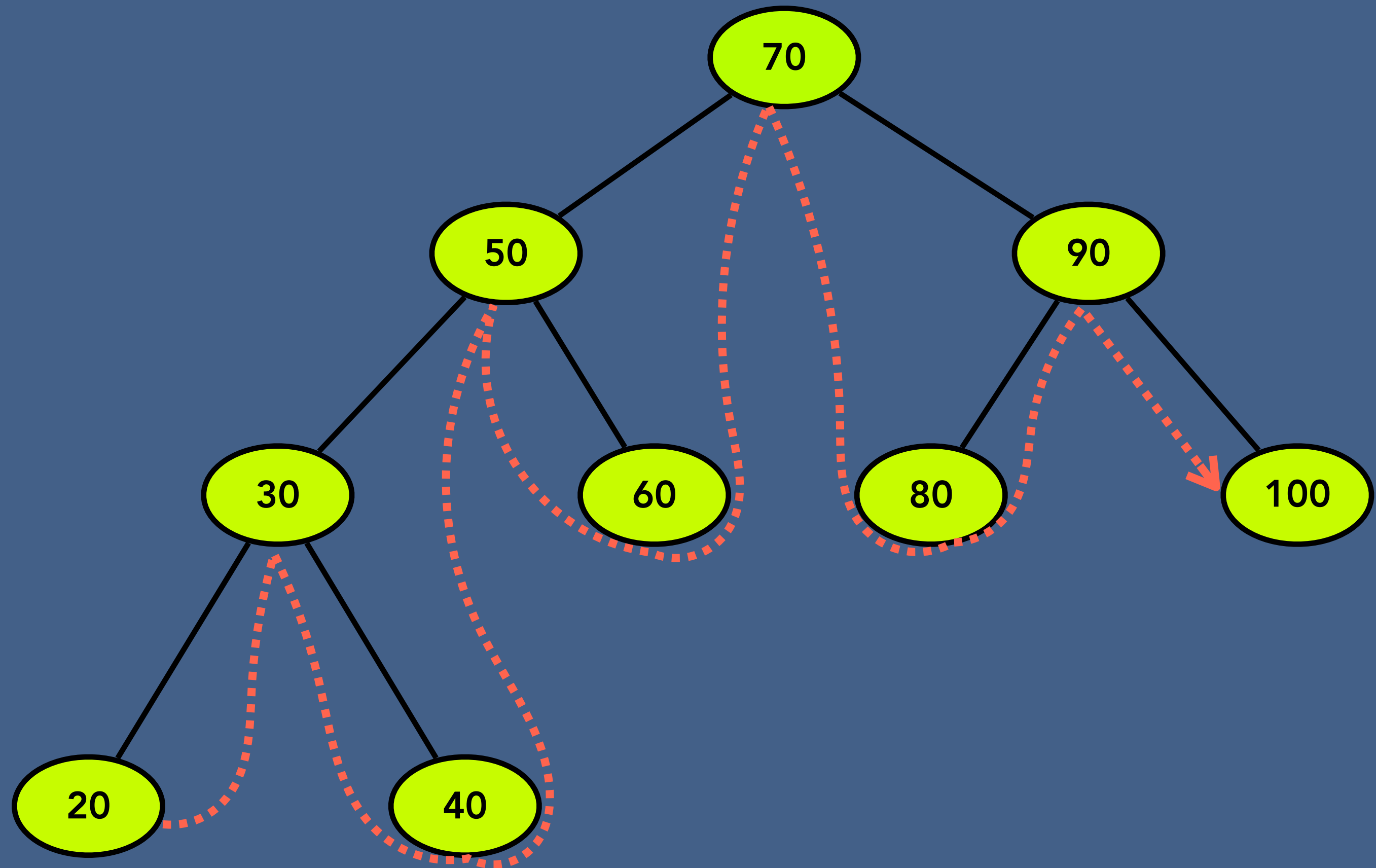
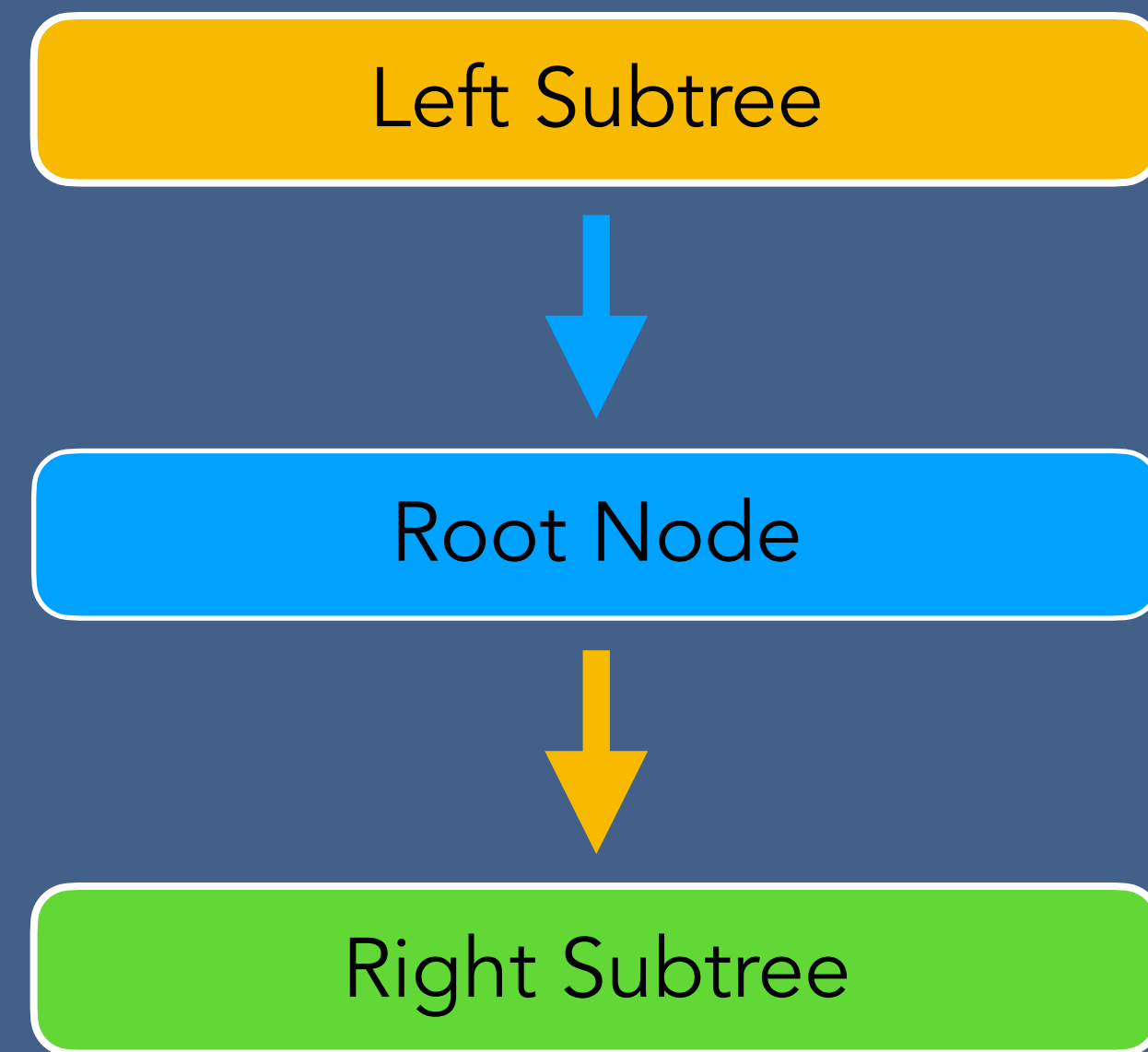
Binary Search Tree - PreOrder Traversal



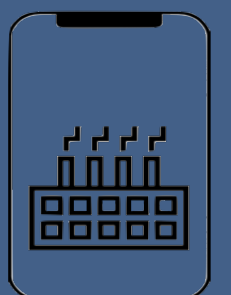
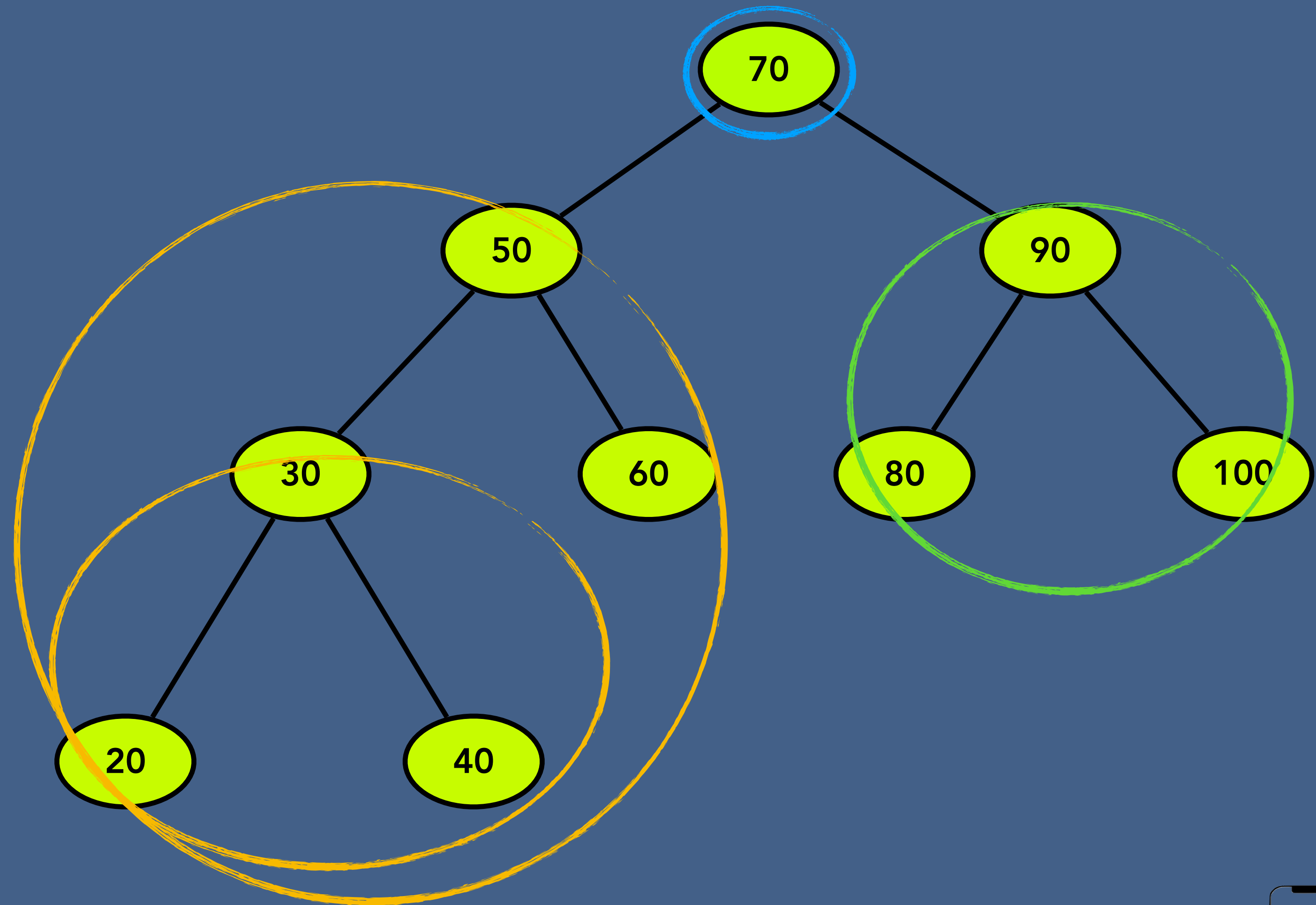
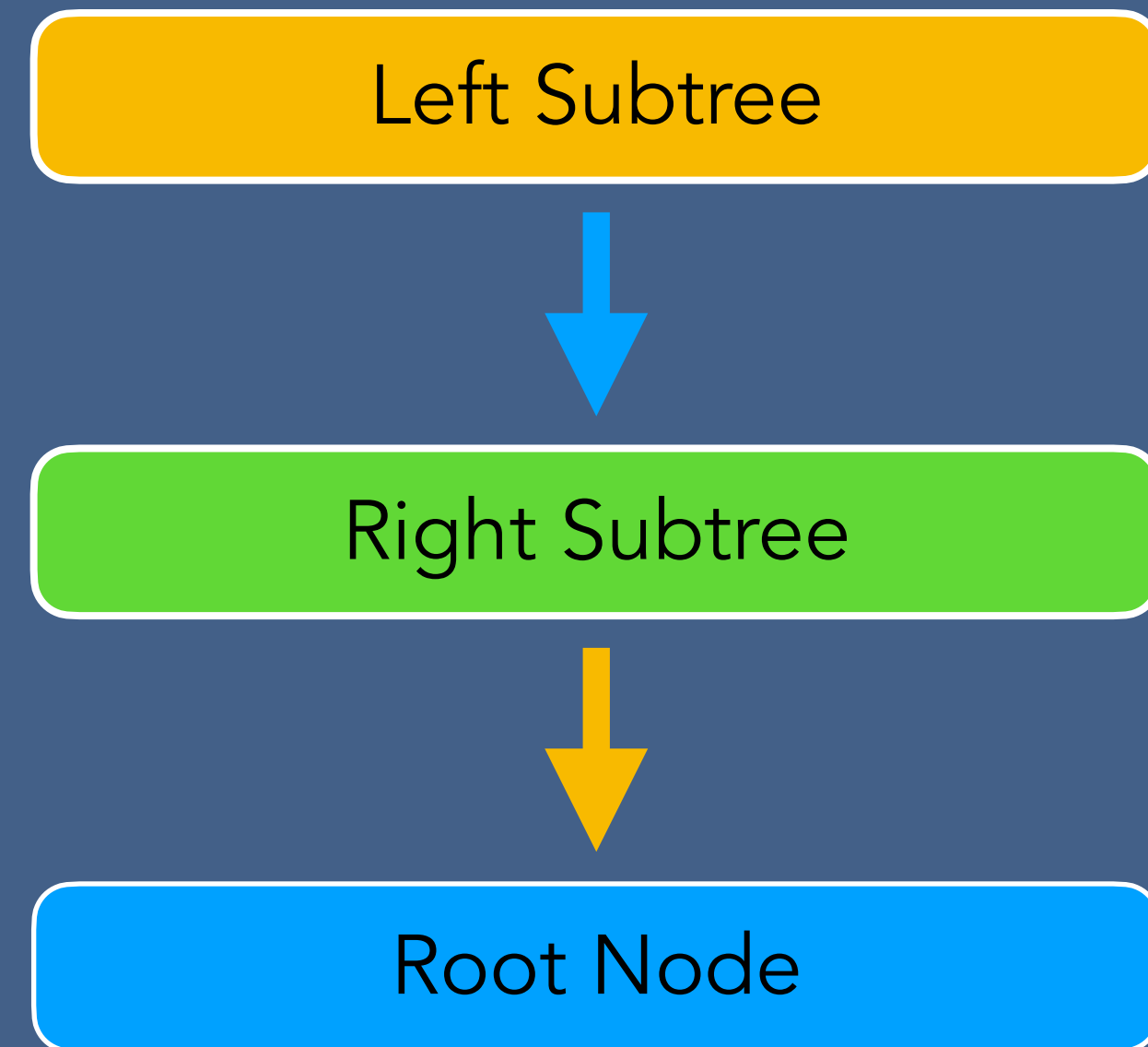
Binary Search Tree - InOrder Traversal



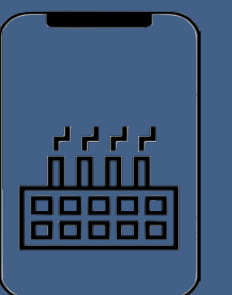
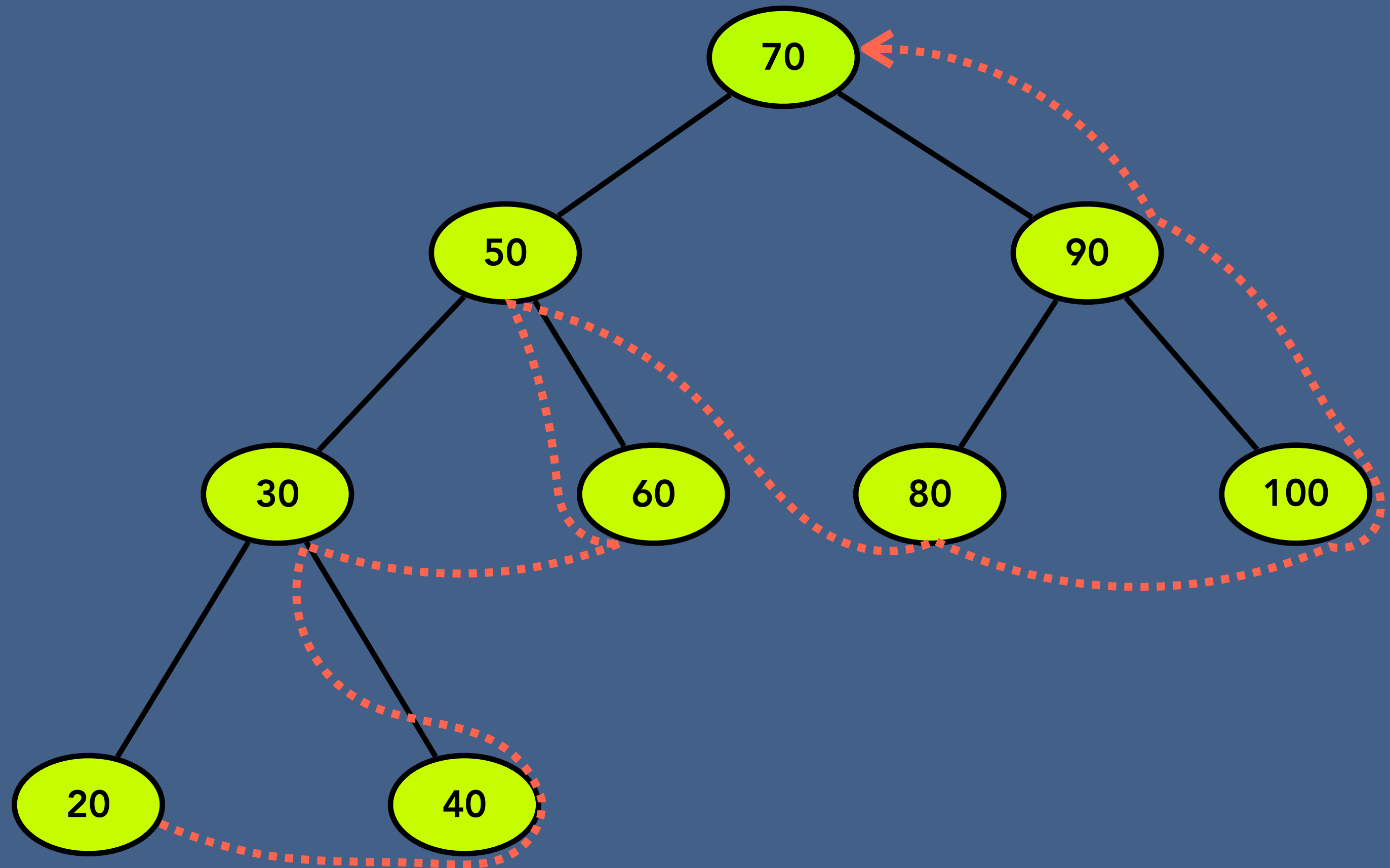
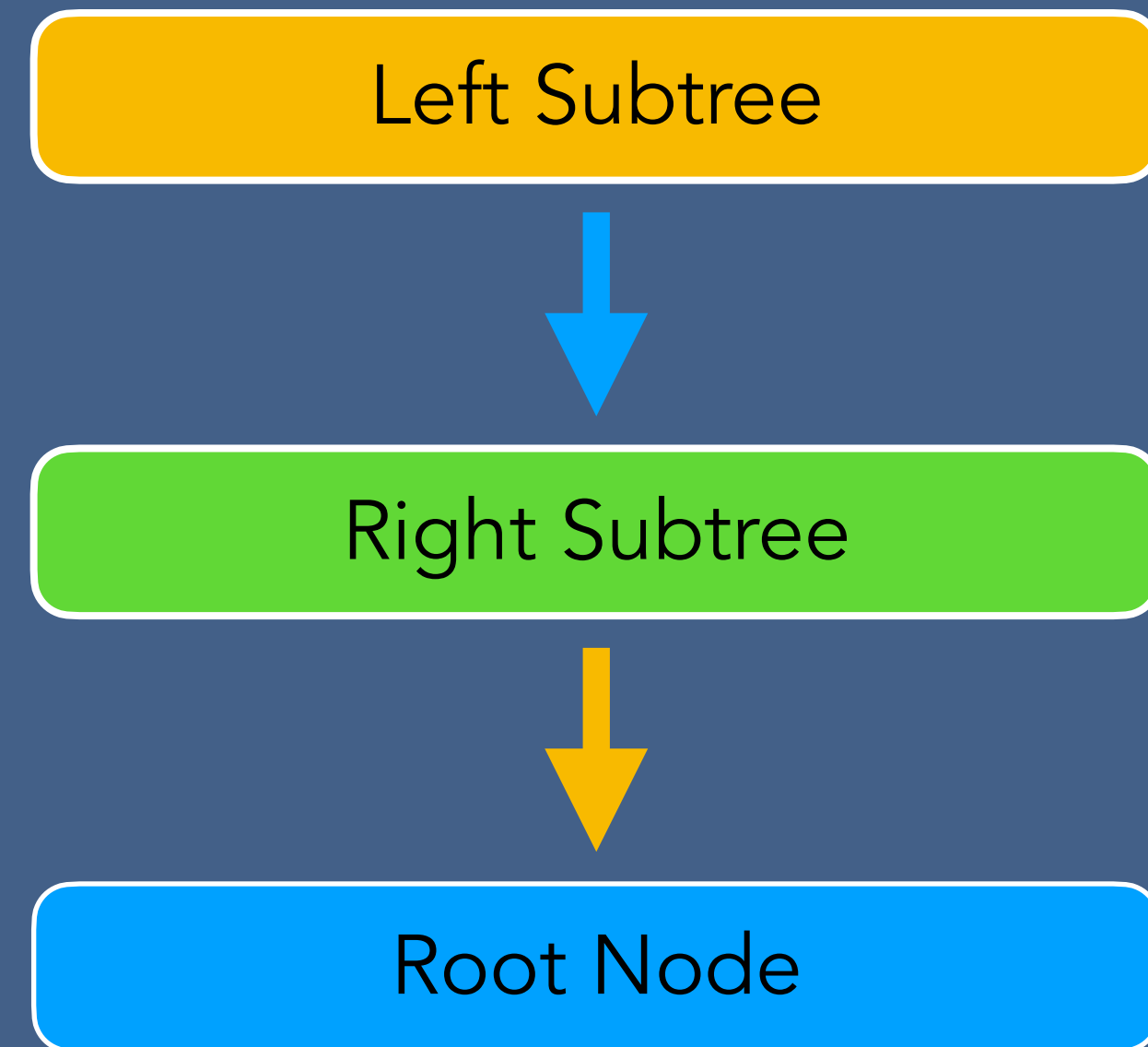
Binary Search Tree- InOrder Traversal



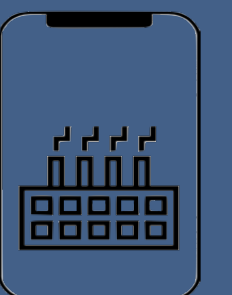
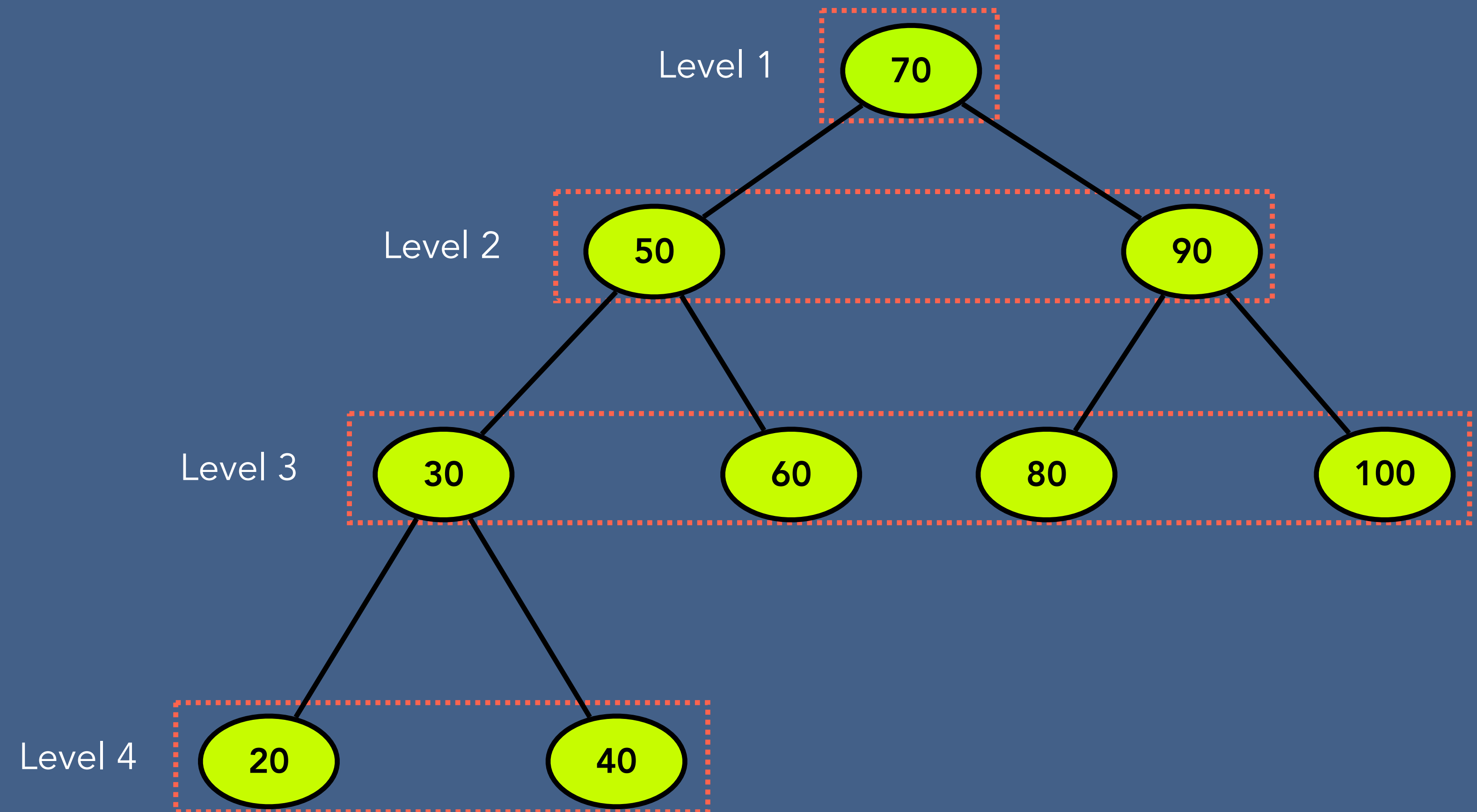
Binary Search Tree- Post Traversal



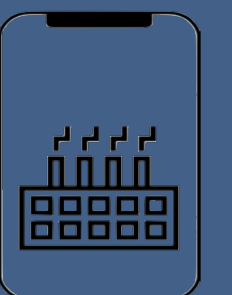
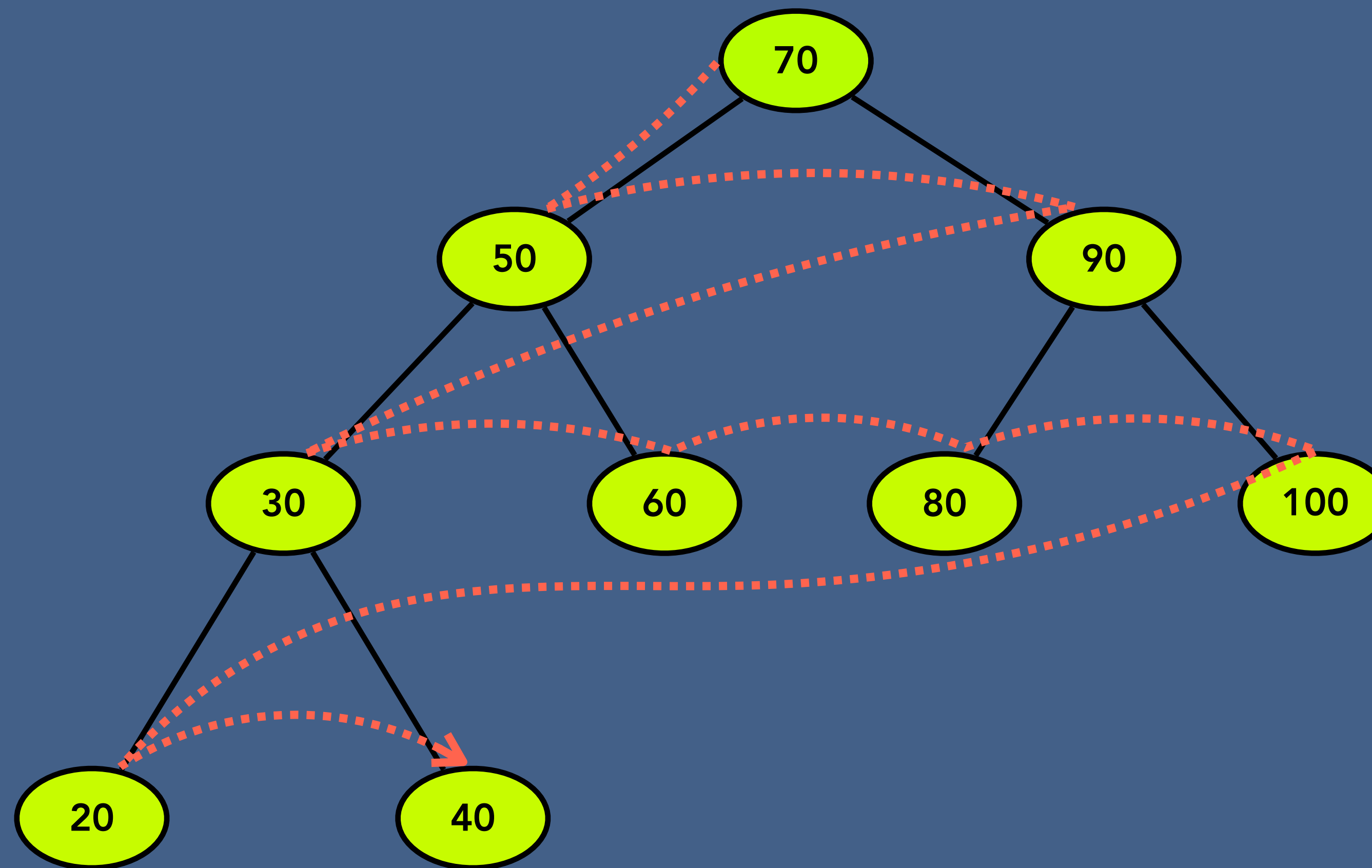
Binary Search Tree - Post Traversal



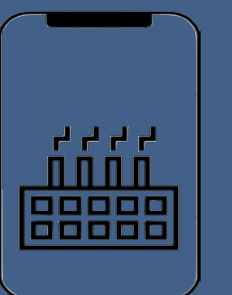
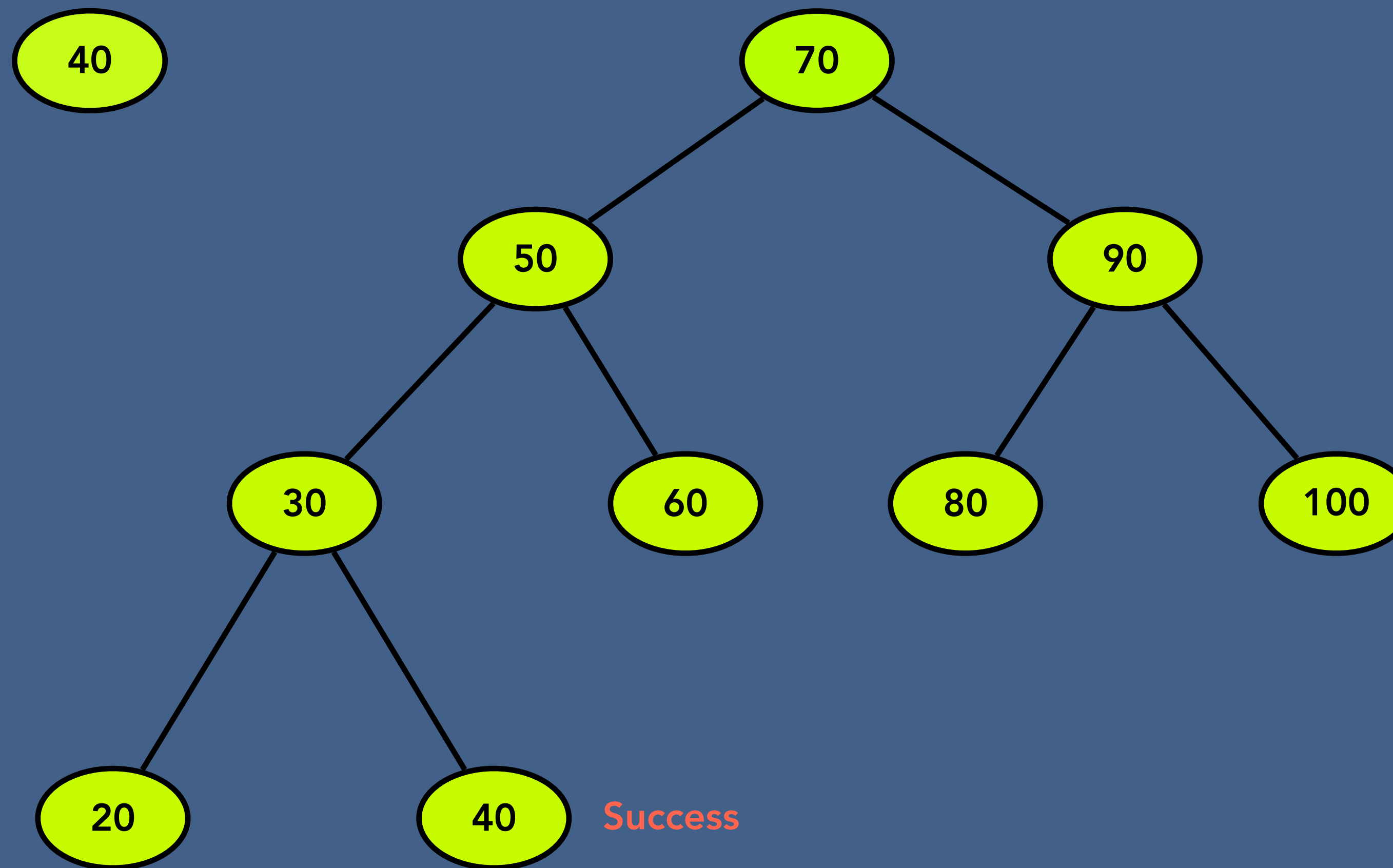
Binary Search Tree - LevelOrder Traversal



Binary Search Tree - LevelOrder Traversal



Binary Search Tree - Search

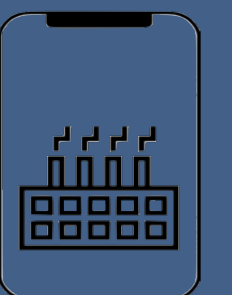
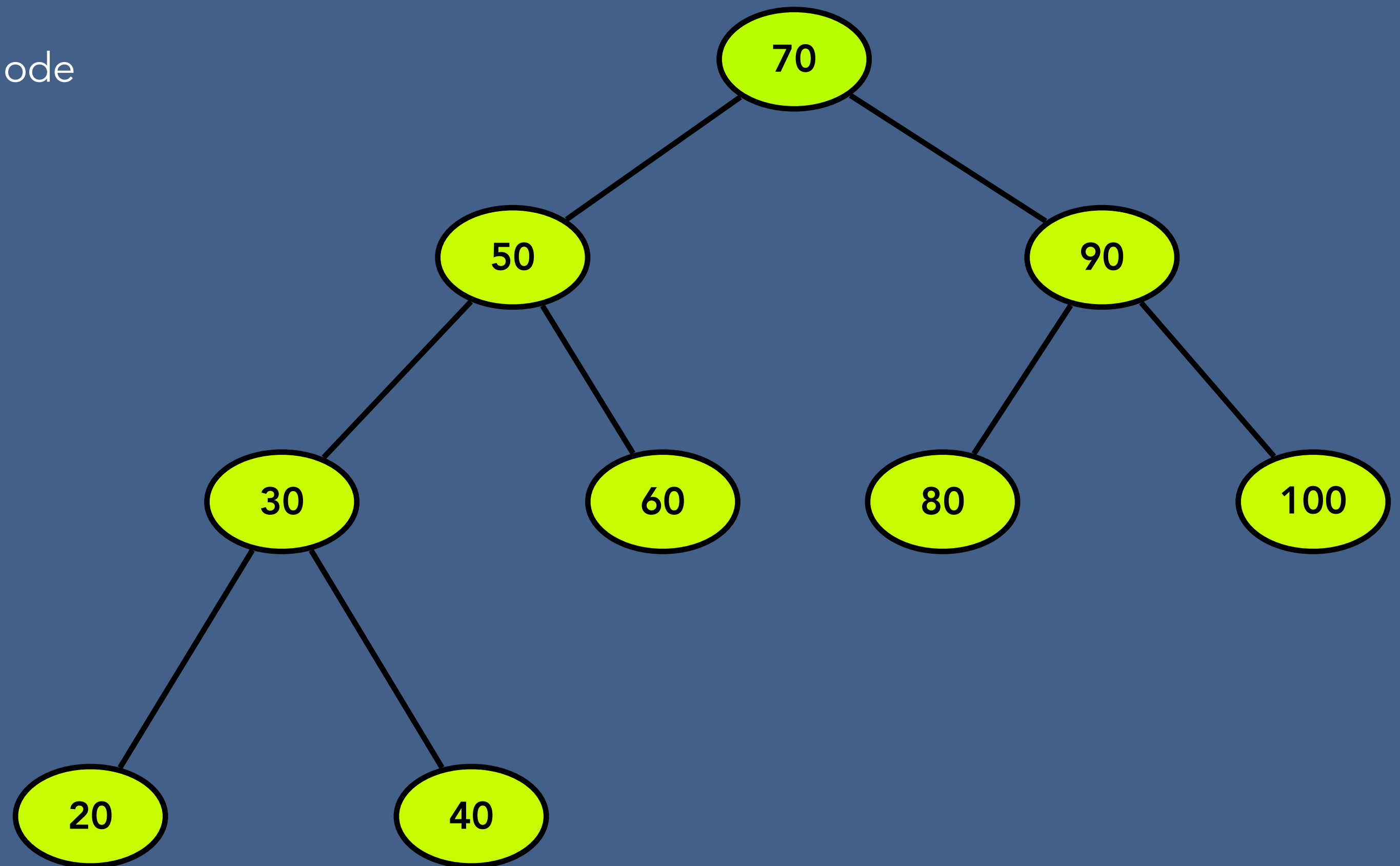


Binary Search Tree - Delete a Node

Case 1: The node to be deleted is a leaf node

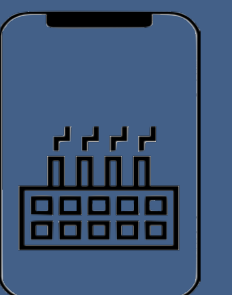
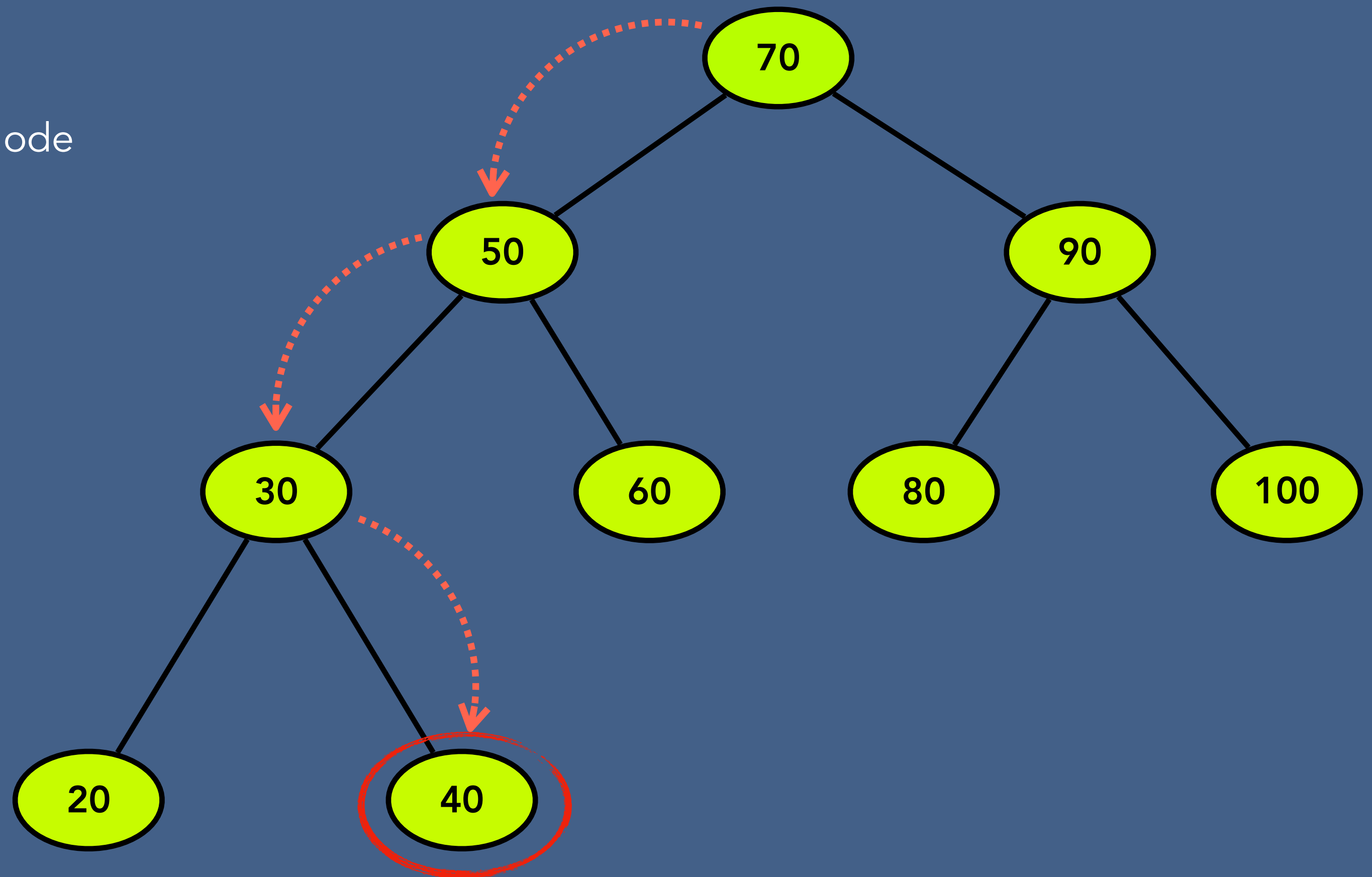
Case 2: The node has one child

Case 3: The node has two children



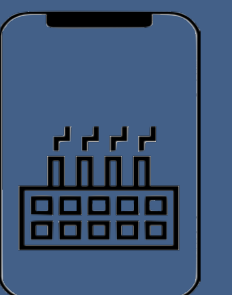
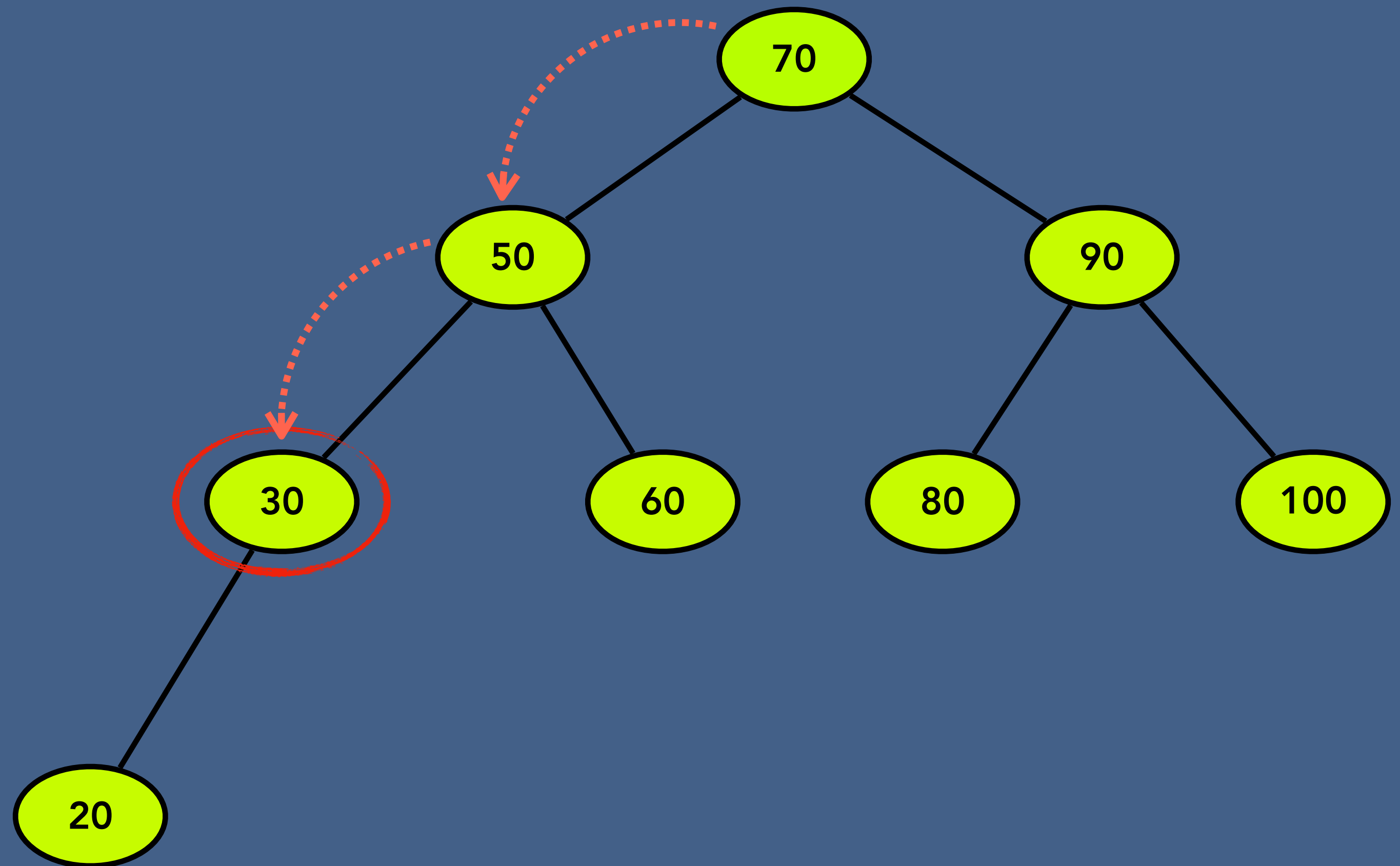
Binary Search Tree - Delete a Node

Case 1: The node to be deleted is a leaf node



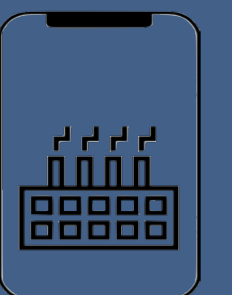
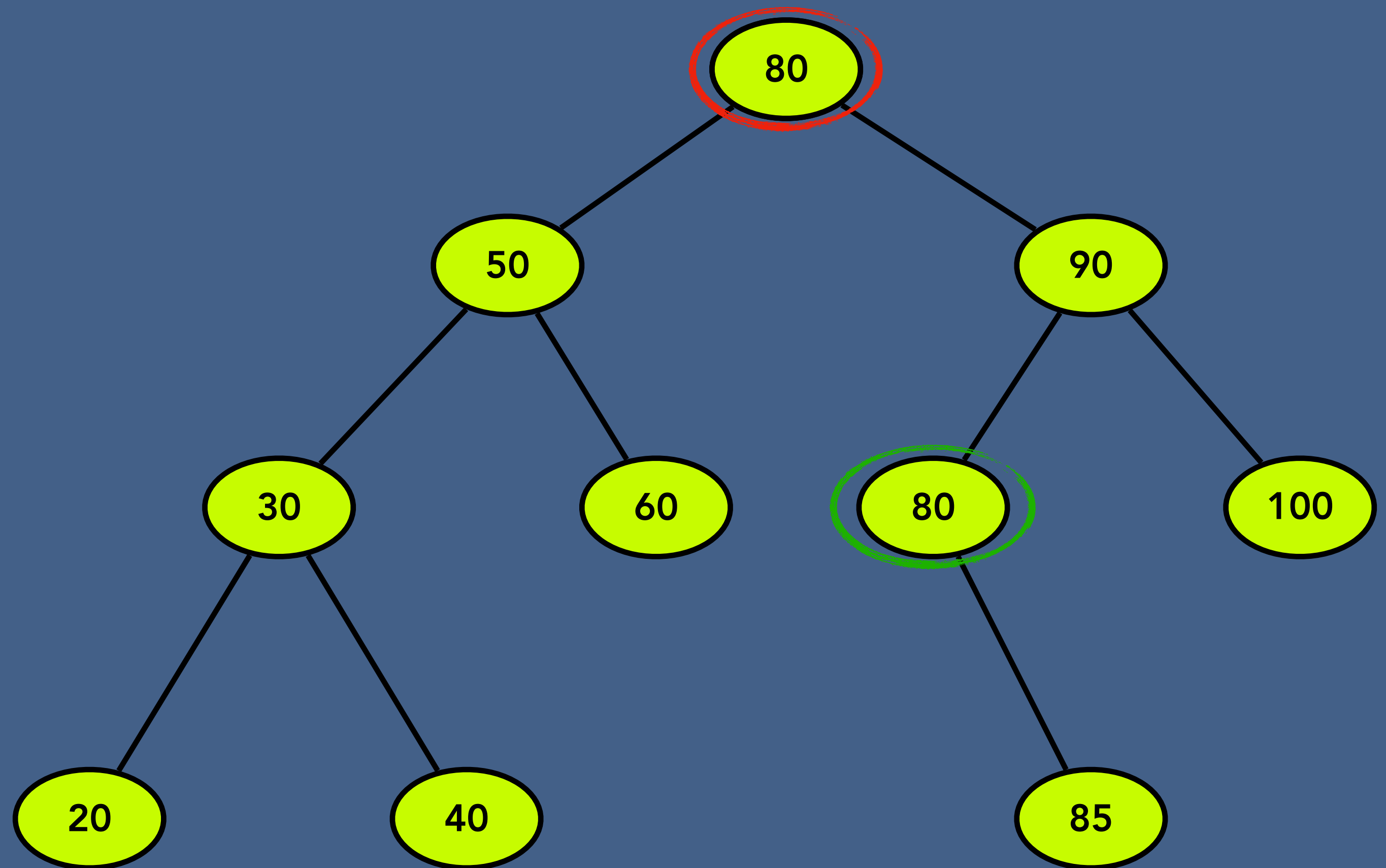
Binary Search Tree - Delete a Node

Case 2: The node has one child



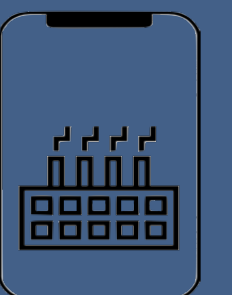
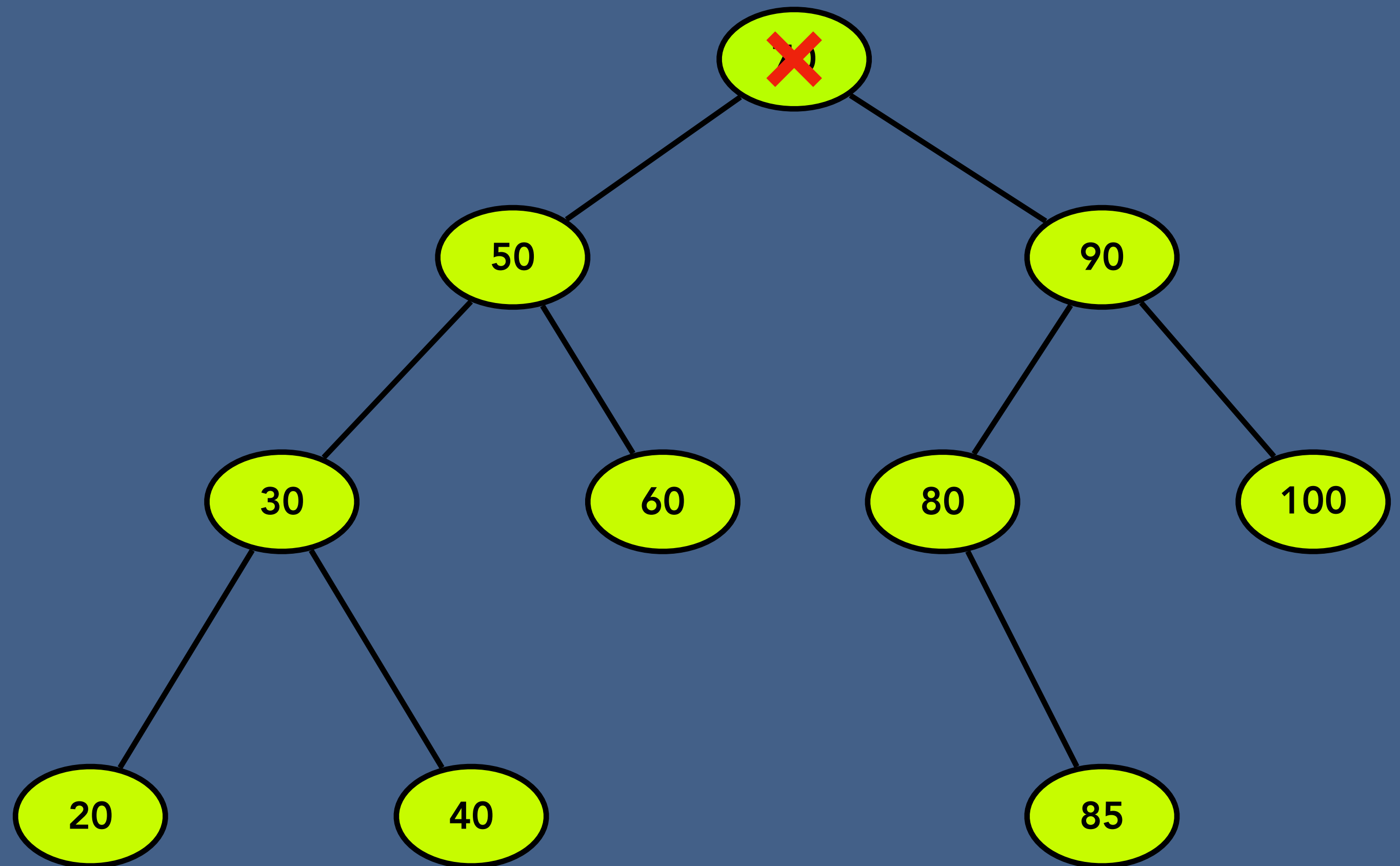
Binary Search Tree - Delete a Node

Case 3: The node has two children



Binary Search Tree - Delete

root = null



Binary Search Tree - Time and Space Complexity

	Time complexity	Space complexity
Create BST	$O(1)$	$O(1)$
Insert a node BST	$O(\log N)$	$O(\log N)$
Traverse BST	$O(N)$	$O(N)$
Search for a node BST	$O(\log N)$	$O(\log N)$
Delete node from BST	$O(\log N)$	$O(\log N)$
Delete Entire BST	$O(1)$	$O(1)$

